

Adaptive security framework for multi-environment networks using ensemble data drift detection and incremental deep learning

Furqan Rustam¹, Anca Delia Jurcut^{2*}

¹School of Computer Science, University College Dublin, Belfield, Dublin 04, D06WK27, Dublin, Ireland

ARTICLE INFO

Dataset link: https://drive.google.com/drive/folders/1Lo7eivyA_7A9-YSQmC-KLQBS0RoLtUP7

Keywords:

Network security
Explainable AI
Multi-environment network
Data drift detection
Incremental learning

ABSTRACT

Modern multi-environment (M-En) networks comprise diverse architectures such as IoT and traditional IP-based networks. These networks pose significant challenges for threat mitigation due to heterogeneous protocols and traffic patterns. This study proposes a unified incremental learning framework to efficiently secure M-En networks by reducing management overhead, improving scalability, and lowering costs. We designed this approach for real-time environments, enabling adaptation to new scenarios with high accuracy and efficiency. To develop the framework, we first generate an M-En dataset using partial least squares canonical analysis, synthesizing data from two benchmark datasets: IoT23 and CICDDoS2019, representing IoT and traditional IP-based networks, respectively. Our approach employs an ensemble data drift detection (EDDD) mechanism that combines ADaptive WIndowing and autoencoders, enabling adaptive model updates. A deep neural network is incrementally retrained only when data drift is detected, ensuring adaptability to evolving attacks while conserving computational resources. To avoid catastrophic forgetting, we incorporate replay-based memory, regularization, and an interpolation mechanism governed by a blending parameter $\alpha \in [0, 1]$, which balances the integration of new and historical knowledge. Furthermore, the explainable AI technique LIME is integrated to enhance the transparency of the model's decision-making process. Experimental results indicate that our approach achieves a mean accuracy of 0.999 while maintaining low memory usage, approximately 32.1 MB, and a stable model size of 0.11 MB.

1. Introduction

In today's digital age, the internet is the backbone of nearly all our activities, from business to education [1]. However, this heavy reliance on network technology also brings significant risks. Cybercriminals exploit vulnerabilities in these systems to steal information, damage reputations, and disrupt services [2]. According to the cybersecurity breaches survey 2025, over 60% of UK businesses experienced a cyber breach or attack in the past year.¹ Geopolitical tensions have also driven a 265% rise in network attacks in 2024, particularly targeting Ukraine, the US, and Israel.² The factor of the growing complexity in network architectures makes more harder to protect systems [3]. Researchers increasingly use machine learning to enhance security [4]. However, these methods still struggle with the diverse traffic in modern networks, as many systems are designed for specific types like IoT or traditional IP-based traffic [5]. A machine learning method trained on specific network traffic may perform well for that network but could struggle to analyze traffic from others effectively [6]. To address this

challenge, the idea of *multi-environment* (M-En) network security has emerged, aiming to protect different network types under a unified framework [2,7].

Traditional network and IoT systems generate different types of traffic in terms of volume, scale, and patterns [8–10]. In traditional networks, malicious traffic typically originates from compromised servers or PCs using fewer, high-capacity devices to generate uniform, high-bandwidth attacks like SYN or HTTP floods. These attacks are usually intense but shorter in duration [11]. In contrast, IoT networks, malicious traffic involves thousands or millions of compromised IoT devices (like cameras or routers) with low-bandwidth connections, creating diverse traffic patterns through multi-vector attacks that can be sustained over longer periods [12]. Analyzing this mix of traffic becomes increasingly challenging as M-En networks grow, where traditional network and IoT technologies coexist in sectors like smart cities, healthcare, and industrial automation. For example, in smart grids, traditional network infrastructure like data centers work alongside IoT devices

* Corresponding author.

E-mail address: furqan.rustam1@gmail.com (F. Rustam).

¹ <https://www.gov.uk/government/statistics/cyber-security-breaches-survey-2025/cyber-security-breaches-survey-2025>

² <https://www.infosecurity-magazine.com/news/geopolitical-tensions-drive-ddos/>

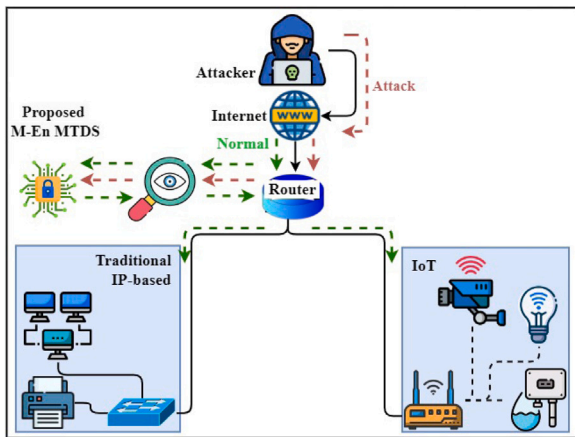


Fig. 1. Real-Time Deployment Scenario of MTDS for M-En.

like smart meters to manage energy distribution efficiently [13]. Similarly, in healthcare, traditional systems integrate with IoT devices for better real-time patient care, making the systems more dynamic and responsive [14].

Numerous studies are focusing on these types of networks; however, they address each network independently, creating separate systems for IoT, traditional networks, etc [15–17]. These independent systems lead to increased complexity, higher management overhead, and additional costs. There are several benefits to implementing a unified system for M-En networks [6]. A unified approach for managing M-En traffic can enhance attack detection capabilities through integrated analysis [18]. Centralized processing improves machine learning models and anomaly detection, while dynamic scalability enables efficient handling of both high-bandwidth traditional attacks and low-bandwidth, large-scale IoT attacks [19]. The system also optimizes resource allocation by applying targeted mitigation strategies based on the specific threat type and simplifies maintenance through unified updates and monitoring. The M-En malicious traffic detection system (MTDS) deployment in the real-time scenario is shown in Fig. 1. This MTDS can be beneficial for both large organizations and small companies, for example, including those with traditional IP-based networks in office environments and warehouses utilizing IoT devices such as network cameras, smart bulbs, and automated doors. Our proposed MTDS can handle both types of networks in a unified manner, ensuring accuracy and efficiency. However, challenges include managing the diverse traffic characteristics and patterns between traditional and IoT environments, as IoT traffic can be more irregular and low-rate [20]. The system must be highly scalable to handle the massive number of connections typical in IoT DDoS attacks. Additionally, designing algorithms that effectively detect both high-intensity traditional attacks and distributed IoT attacks without introducing latency is complex. Balancing performance, real-time analysis, and resource management without causing bottlenecks remains a significant challenge.

The lack of a comprehensive benchmark dataset for M-En networks has been a major challenge in developing an effective unified system capable of handling both traditional and IoT network traffic. Existing approaches to generating M-En datasets often fail to capture the diverse traffic patterns and attack vectors characteristic of real-world M-En networks. To address this, our previous research proposed integrating datasets from IoT and traditional IP-based networks to create a more accurate representation of M-En traffic [5,6]. However, those approaches had limitations, particularly in adapting to dynamic changes in network traffic over time and in providing explainability for the detection process. To overcome these limitations, the current research incorporates data drift detection techniques that monitor and adjust for shifts in network traffic patterns in real time. By integrating incremental

learning, the system continuously updates its models as traffic evolves, ensuring that detection algorithms remain effective under changing network conditions. Furthermore, to enhance transparency and trust in decision-making, *explainable AI* (XAI) methods are employed to clarify the reasoning behind the model's decisions, making it easier to understand and justify actions in complex scenarios. This combination of adaptive learning and explainability offers a robust solution for maintaining security in dynamic M-En networks. Our contributions are summarized below:

- In this study, we proposed an approach for real-time network security against malicious traffic, which can adapt to new scenarios and environments even after deployment, while maintaining high accuracy and efficiency.
- In this study, we generated an M-En dataset using two benchmark datasets, IoT23 and CICDDoS2019, along with *Partial Least Squares* (PLS) canonical analysis. This M-En dataset helped us to propose a framework that can work on diverse network scenarios with a unified approach.
- We introduced a data drift detection approach for network traffic in real-time scenarios using an ensemble of deep learning and statistical methods. The *Ensemble Data Drift Detection* (EDDD) method combines an autoencoder and the ADWIN method, with final results determined using an 'OR' operation. The EDDD is capable of detecting changes in traffic in real-time scenarios. If there is a change or new traffic, our proposed MTDS will automatically adapt to the new traffic patterns.
- We propose an incremental DNN-based learning approach triggered by EDDD, and evaluate replay, regularization, and interpolation (with blending factor $\alpha \in [0, 1]$) to mitigate forgetting. Weight changes are monitored to assess model stability under evolving traffic conditions.
- We employed the XAI approach using LIME with two methods: linear representation and force plot, to identify which factors contribute to the model's predictions in individual cases and within the M-En environment, thereby interpreting the black-box decisions.
- We evaluated the performance of the proposed framework in terms of memory usage, model size, and changes in model weights over time.

Further, this paper is organized as follows: Section 2 presents the literature review, Section 3 outlines the proposed methodology, Section 4 present experimentation setup, Section 5 contains the results & discussion, and Section 6 provides the conclusion.

2. Related work

Researchers have proposed various machine learning approaches to secure modern network architectures [4]. This section reviews recent literature on M-En network security, incremental learning, and explainable AI, and outlines the research gap our study addresses.

2.1. M-En network security

This section highlights recent research on security challenges, with a focus on M-En networks. Optimizer-based approaches enhance security in M-En networks; for instance, [2] employed an ensemble learning method optimized using *particle swarm optimization* (PSO). They used the M-En network dataset, which comprises traffic from IoT, traditional IP-based, and SDN-based networks. Various techniques, including feature selection and data balancing, were used to preprocess the M-En dataset. The proposed PSO-based diverse self-ensemble model achieved a significant accuracy of 0.989. In another study, a fully automated system (FAMTDS) utilizes the *Moth Flame Optimizer* (MFO) and feature engineering approaches to tackle the M-En [6]. A neural network is deployed to generate features, followed by the use of MFO for optimizing

and training the model. Their proposed approach achieved a significant accuracy of 0.85 across 19 target attacks.

Several studies have focused on data balancing and synthetic data generation approaches to address M-En diversity; for example, [21] tackled security in both IoT and traditional IP-based M-En networks. They utilized two datasets, IoTID20 and UNSWNB15, to generate the M-En dataset. To address dataset imbalance and reduce model overfitting, they introduced a *synthetic data augmentation technique* (S-DATE). Their results showed that the *Random Forest* (RF) model outperformed other machine learning models, achieving an accuracy score of 0.991. In another study [22], AI-generated attacks in M-En networks were addressed by proposing an approach that leverages generative models such as CTGAN, CopulaGAN, and TVAE to generate synthetic AI-based traffic. They aimed to detect both AI-based and human-generated attacks by improving generalization through training on diverse traffic patterns.

A study explored traffic adaptability in modern M-En networks; [23] proposed an adaptive framework to secure SDN-enabled IoT networks against DDoS attacks. Their system integrates a layered classification approach, where models like *Support Vector Machine* (SVM), Naive Bayes, RF, *k-nearest Neighbor* (kNN), and *Logistic Regression* (LR) are initially trained and then combined through an Ensemble Voting mechanism to enhance detection accuracy. The adaptive framework monitors real-time network traffic and dynamically updates its model using SDN controllers that manage OpenFlow switches.

2.2. Incremental learning in network security

This section reviews literature on incremental learning in network security, a growing need as network traffic evolves with advancing architectures. The study [24] proposed an incremental learning approach named *tree deep feedforward neural networks* (T-DFNN) for network attack detection. T-DFNN-based incremental learning adapts to new attacks without full retraining by using a hierarchical structure of T-DFNNs, reducing catastrophic forgetting. They evaluate T-DFNN using the CICIDS2017 dataset and demonstrate that T-DFNN achieves an F1-score above 0.85 across retraining processes, with faster training times. Similarly, the study [25] introduced a *dynamic ensemble incremental learning approach based on relevance vector machines* (DEIL-RVM). Unlike traditional static models, DEIL-RVM addresses concept drift by dynamically updating its RVM ensemble with adaptive voting, enabling efficient learning from new data while maintaining high accuracy. The study performs experiments on the KDDCUP99 and CICIDS2017 datasets and achieves 0.994, outperforming other ensemble models like Bagging-SVM and Learn++.NSE. Another study [26] also adopts an error-rate-based drift detection technique and incrementally trains LSTM-based models. Their proposed method, called *concept drift detection and adaptation-based malicious traffic detection* (CDDA-MD), demonstrates significant results on specific datasets and illustrates how incremental learning, in combination with a drift detection mechanism, can improve performance.

Recent advances in network attack detection have increasingly leveraged *Class Incremental Learning* (CIL) [27]. The study [28] introduces an adoption-based approach that updates models with new traffic behaviors without storing past data, achieving a 32% faster adaptation to new attacks using the UNSWNB15 dataset. Similarly, [29] proposes *Hierarchical Federated Incremental Learning* (HFIN) for IIoT environments, combining federated and CIL with a Weighted Transmission Optimization strategy. Evaluated on NFUQNIDSv2 and NFToIoTv2, HFIN improved accuracy and F1-score by 10% over baselines, highlighting its adaptability and privacy-preserving capabilities. Another study [30] proposed a few-shot class-incremental learning method called *Branch Fusion Strategy-based Network Intrusion Detection* (BFS-NID) to address the challenge of detecting new and evolving attack types with minimal labeled samples while avoiding catastrophic forgetting of previous classes. The BFS-NID framework consists of a self-supervised vision

transformer-based feature extractor and a branch classifier module. The method was evaluated on two benchmark datasets, CICIDS2017 and CICIDS2018, and experimental results showed that BFS-NID significantly outperforms baseline methods, achieving accuracy scores of 0.93 and 0.90 for each dataset, respectively.

2.3. Explainable AI in network security

Explainable AI (XAI) is increasingly used in MTDS to enhance transparency and address the black-box nature of AI models, improving interpretability and trust [31]. The study [32] developed a deep learning-enabled IDS for IoT networks, leveraging XAI to improve the clarity and accountability of model predictions, which is essential for real-world deployment in sensitive environments, and used SHAP & LIME for global and local interpretability. Another critical contribution by the study [33] involved training FGSM, PGD, and DeepFool models against adversarial attacks and using SHAP for explainability, which provided both local and global interpretability. In the automotive domain, the study [34] explored the use of XAI in developing a reliable in-vehicle IDS. They introduced a visualization-based explanation method called VisExp, built using SHAP, to interpret the behavior of a DNN-based in-vehicle IDS.

The study [35] proposed the XAI-IDS framework used in NIDS, which also addresses the challenges posed by the black-box nature of AI models. The proposed system integrates XAI techniques like SHAP and LIME to generate both local and global explanations, offering insights into the features influencing AI decisions. The framework was evaluated using three datasets: CICIDS2017, NSLKDD, and RoEduNet-SIMARGL2021. Experimental results show that the XAI-IDS framework provides robust explanations while maintaining high detection performance. Similarly, the study [36] explores a framework integrating XAI techniques into deep learning models to enhance the transparency and interpretability of IDS for IoT networks. The authors propose a deep learning-based approach utilizing both a DNN and a *convolutional neural network* (CNN) to classify network traffic into normal or various attack categories. Their study employs the NSLKDD and UNSWNB15 datasets to evaluate the models, and it also deploys LIME and SHAP to provide local and global model interpretability.

2.4. Feature engineering in network security

Feature selection and reduction are vital in network security, with techniques like PCA, SVD, Gini index, and chi-square commonly used for preprocessing. In our study, we used PLS to map IoT and traditional datasets into a shared feature space, forming the M-En dataset. PLS effectively reduces dimensionality while preserving relevant information, improving malicious traffic detection performance. The study [37] proposed a PLS feature extraction approach with a *Core Vector Machine* (CVM) to enhance attack detection accuracy. The study addressed high-dimensional network data by using PLS to extract key features relevant to intrusions, reducing noise and enabling the CVM classifier to focus on essential information. Similarly, in study [38], the author explored the application of *PLS Discriminant Analysis* (PLS-DA) within a compositional data analysis framework for security applications. The research highlights the use of PLS-DA to address the inherent correlations and dependencies among features in compositional data, commonly encountered in cybersecurity settings. PLS-DA method enables the transformation of the feature space into canonical components, which are then used for classification and detection. Some studies also used statistical techniques for feature engineering, such as the study [7], which utilized PCA to reduce the feature space of the IoTID20 and UNSWNB15 datasets simultaneously, allowing them to combine these datasets to generate an M-En dataset. Similarly, another study [39] proposed an LSTM-based IDS and utilized linear and non-linear feature reduction techniques such as PCA and autoencoder to generate a fused feature set, which improved the performance of

Table 1
Summary of related work.

Ref	Year	APR	MEnS	Dataset	Model	MTD	IL	RTA	GEN	HA	XAI	LW
[24]	2021	★	○	CICIDS2017	T-DFNN	IL	●	○	○	●	○	○
[25]	2021	◆	○	KDDCUP99, CICIDS2017	DEIL-RVM	DEL	○	○	●	●	○	●
[7]	2022	◆	○	IoTID20, UNSWNB15	SE	PCA-based FS	○	○	●	●	○	●
[23]	2022	◆	○	SDN-IoT	EV	LC	○	○	○	●	○	○
[34]	2022	★	○	VN	DNN	DL-XAI	●	○	○	●	●	○
[32]	2023	★	○	IoT	LSTM, RNN	DL-XAI	○	○	●	●	●	○
[30]	2024	★	○	CICIDS2017, CICIDS2018	BFS-NID	FSIL	●	●	○	●	●	●
[29]	2024	◆	○	NFUQNIDSv2, NFToNIoTv2	HFIN	FIL	●	○	●	●	○	○
[28]	2024	◆	○	UNSWNB15	RF	IL-Adoption	●	●	○	●	○	●
[35]	2024	★	○	CICIDS2017, NSLKDD99, RoEduNet-SIMARGL2021	LightGBM, KNN, SVM, SHAP, LIME	DL-XAI	○	○	●	●	●	○
[36]	2024	★	○	NSLKDD99, UNSWNB15	DNN, CNN	DL-XAI	○	○	●	●	●	○
[39]	2024	★	○	NSLKDD99, UNSWNB15	LSTM	PCA & AF	○	●	○	●	○	○
[2]	2024	◆	●	CICIDS2017, MSCAD, IoTID20, UNSWNB15 InSDN	PSO-D-SEM	FS & DB	○	○	○	●	○	●
[21]	2024	◆	●	IoTID20, UNSWNB15	RF	S-DATE, DB	○	●	○	●	○	●
[22]	2024	◆	●	IoTID20, UNSWNB15 InSDN	ETC	CTGAN, CopulaGAN TVAE	○	○	○	●	○	○
[6]	2024	★	●	IoTID20, UNSWNB15	NN, RF	MFO	○	●	○	●	○	●
[26]	2025	★	○	USTCTFC2016, CICIDS2017, ISAC218, UJSIDS2024	LSTM	MHA DD	●	●	○	●	○	○
Our	2025	★	●	IoT23, CICDDoS201	DNN	FS & EDDD	●	●	●	●	●	●

APR—Approach; MTD—Methods; IL— Incremental Learning; RTA— Real-Time Traffic Adaptability; GEN— Generalization in diverse environments; HA— High Accuracy; MEnS— Multi-Environment Network Support; LW— Lightweight Model (low resource use); ... **Acronyms:** Feature Selection (FS); Data Balancing (DB); Ensemble Voting (EV); Layered Classification (LC); Dynamic Ensemble Learning (DEL); Federated Incremental Learning (FIL); Vehicular Networks (VN); Deep Learning and Explainable AI (DL-XAI); Stacked Ensemble (SE); Autoencoder Fusion (AF); AdaBoost + Hoeffding Tree (AB-HT); Rethinking Few-Shot in Few-Shot Class Incremental Learning (RFS-FSCIL); Incremental & Interpretable Recurrent Neural Network (I^2 RNN); Multi-Head Attention (MHA); Drift Detection (DD) ... **Symbols:** ● Present, ◐ Partial, ○ Lacking, ◆ ML, ★ DL

machine learning algorithms. Their approach was deployed on the NSLKDD99, UNSWNB15, CICIDS2017, and MSCAD datasets, resulting in enhanced performance compared to previous studies. Table 1 summarizes the related work, including methods, datasets, and other comparative analytics.

Limitation & Gap: Several recent studies have employed state-of-the-art approaches in M-En networks as well as in specific network types. However, these studies often rely on outdated datasets such as IoTID20 and UNSWNB15 [5], which may not adequately represent recent attacks [6]. Another limitation in recent research is the neglect of new traffic patterns and the failure to adopt new environments using incremental learning [22]. While some studies have used incremental learning for MTDS, they have not applied it in M-En networks [24]. Moreover, the incremental learning approaches used often involve continuous model retraining, making the models more complex and resource-intensive [30], and their catastrophic issue is not deeply analyzed. Lastly, XAI adoption is missing in many M-En studies because of the complexity in features and involvement of different protocols, although it is crucial for improving the trustworthiness of proposed approaches in M-En networks [2,7]. Considering these limitations, our work focuses on recent attacks and diverse network traffic patterns. We incorporate incremental learning but limit retraining to when it is truly needed, reducing cost and complexity. Additionally, to enhance trustworthiness in M-En MTDS, we deploy XAI techniques.

3. Proposed methodology

This study proposes a unified, incremental learning framework for real-time malicious traffic detection in M-En networks, as shown in Fig. 2. The proposed framework helps in reducing computational cost and management overhead while adapting to evolving attacks with minimal effort.

Our approach utilizes an M-En dataset synthesized by integrating IoT-specific and traditional IP-based traffic datasets. This integration is facilitated through *feature selection* (FS) and feature decomposition

techniques. FS uses the *Extra Trees* (ET) classifier, which ranks features based on their contribution to classification accuracy. Specifically, the importance score for a given feature f_i is calculated as $I(f_i) = \frac{1}{n} \sum_{j=1}^n \Delta A_j(f_i)$, where n is the number of trees in the ensemble and $\Delta A_j(f_i)$ indicates the improvement in accuracy contributed by feature f_i in the j th tree. After FS, we apply PLS cross-decomposition to extract correlated feature spaces, represented as $\mathbf{X}_{\text{corr}} = \text{PLS}(\mathbf{X}, \mathbf{Y})$, where $\mathbf{X}$ and $\mathbf{Y}$ are the feature matrices derived from the IoT and traditional datasets, respectively. The resulting M-En dataset is labeled with two classes, Attack and Benign, corresponding to IoT-specific and traditional network traffic (see Section 3.1 for more details). Following this, our incremental learning framework employs a DNN that is continuously trained on the evolving M-En dataset. The system integrates an intermediate processing device positioned between the IoT and traditional network environments, as illustrated in Fig. 1. Incoming traffic is first preprocessed and then analyzed by the EDDD mechanism, which combines ADWIN and an Autoencoder. This mechanism determines whether the data is new or previously seen by the DNN. Drift detection is triggered based on the combined output of ADWIN and the Autoencoder:

$$\text{Drift} = \begin{cases} 1, & \text{if } \text{ADWIN}(\mathbf{X}_{\text{new}}) > \tau_a \text{ or } \text{Autoencoder}(\mathbf{X}_{\text{new}}) > \tau_b \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

where τ_a and τ_b are the drift detection thresholds for ADWIN and the Autoencoder, respectively, and $\mathbf{X}_{\text{new}}$ denotes the incoming data. Upon drift detection, the DNN undergoes incremental retraining with the new data while preserving previously learned knowledge, as defined by:

$$\mathbf{W}_{\text{new}} = \alpha \mathbf{W}_{\text{old}} + (1 - \alpha) \mathbf{W}_{\text{update}} \quad (2)$$

Here, $\mathbf{W}_{\text{new}}$ are the updated weights, $\mathbf{W}_{\text{old}}$ are the existing model weights, and $\mathbf{W}_{\text{update}}$ are the weights learned from the new data. The parameter $\alpha \in [0, 1]$ controls the trade-off between retaining historical knowledge and integrating new information. In addition, we use replay memory and regularization methods to address the challenges of retaining and forgetting weight policies in DNN models.

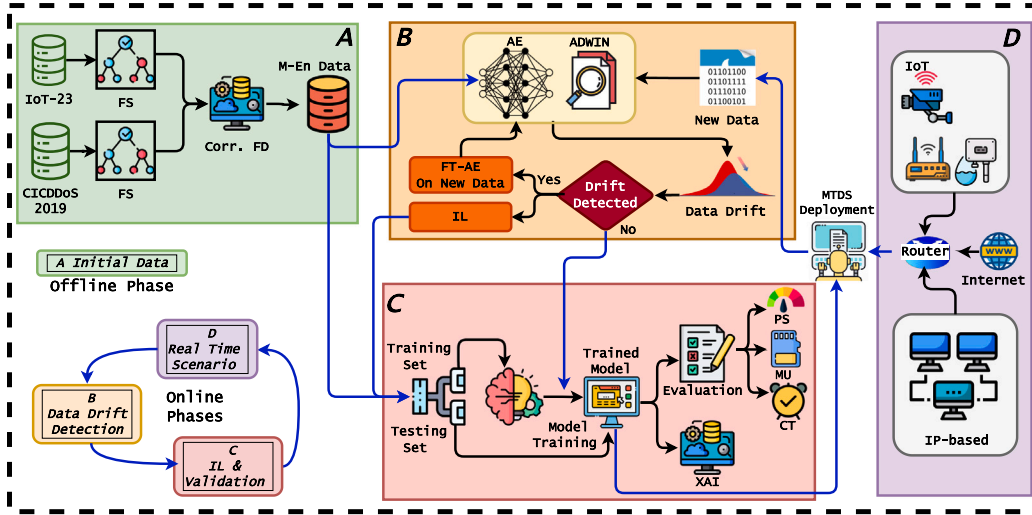


Fig. 2. Proposed MTDS Framework Diagram.

3.1. M-En dataset

Dataset Composition: The M-En dataset integrates samples from the IoT23 [40] and CICDDoS2019 [41] datasets, where each dataset is represented as $X_{IoT} \in \mathbb{R}^{m \times p}$ and $Y_{Ir.IP} \in \mathbb{R}^{n \times q}$, respectively. The CICDDoS2019 dataset features capture critical aspects of network traffic, including packet sizes, flow durations, connection states, and specific protocol-level flags such as ACK, RST, and PSH. For instance, features like *Flow Duration* and *Packet Length Mean* provide insights into traffic flow characteristics, while protocol-based features like *proto_tcp* and *proto_udp* differentiate traffic types. Similarly, the IoT23 dataset features include timestamp (*ts*), origin and response packet counts (*orig_pkts*, *resp_pkts*), and specific connection states (*conn_state_SF*, *conn_state_REJ*) that are indicative of IoT-specific behaviors and communication patterns.

Methodology for M-En Dataset Generation: The dataset generation follows a multi-step process. First, FS is applied using the ET classifier, selecting features S_{IoT} and $S_{Ir.IP}$ based on importance scores $\theta > 0.001$. The resulting feature sets $S_{IoT} \in \mathbb{R}^{50000 \times 19}$ and $S_{Ir.IP} \in \mathbb{R}^{50000 \times 43}$ are then aligned using PLS canonical analysis. The alignment process involves minimizing the Frobenius norm $\|X_{IoT} - Y_{Ir.IP}R\|_F$ to find the optimal transformation matrix R , as formulated in Eq. (3).

$$R = \arg \min_{R \in \mathbb{R}^{d \times d}} \|X_{IoT} - Y_{Ir.IP}R\|_F \quad (3)$$

Next, *Correlation Analysis* (PLS) is performed, computing cross-covariance matrices $C_{XX'} = \frac{1}{n} X_{IoT}^T X_{IoT}$ and $C_{YY'} = \frac{1}{n} Y_{Ir.IP}^T Y_{Ir.IP}$, followed by Singular Value Decomposition on $C_{XX'} = U \Sigma V^T$, where canonical vectors in U and V represent the final merged feature space for the M-En dataset. For balanced experimentation, we extracted 50,000 samples from each dataset and then combined them as shown in Fig. 3.

3.2. Ensemble Data Drift Detection (EDDD)

The EDDD method integrates ADWIN and an autoencoder to detect drift from both statistical and deep learning perspectives, triggering retraining only when either detects drift, thus minimizing unnecessary updates and computational overhead. In our approach, ADWIN is a data stream change detection algorithm that adapts to changing conditions in real-time by dynamically adjusting the size of its sliding window [42]. ADWIN's primary objective is to detect changes or drifts in the average value of a data stream over time. On the other hand, the autoencoder is a deep learning model pre-trained on historical data

X_{hist} . When new data X_{new} arrives, the autoencoder attempts to reconstruct it using the patterns learned from the historical data [43,44]. If the reconstruction error is high, it indicates that the new data is significantly different from the historical data, signaling a potential concept drift. Let, (i) X_{new} represent the new batch of network traffic data. (ii) X_{hist} represent the historical network traffic data.

ADWIN-Based Approach: ADWIN is a statistical method, used in our EDDD framework, that efficiently detects data drift by evaluating deviations between X_{new} and X_{hist} . ADWIN helps to compute whether there is drift by following these steps: (i) First, ADWIN computes the mean of the historical data and the new data as formulated in Eq. (4),

$$\mu_{hist} = \frac{1}{n_{hist}} \sum_{i=1}^{n_{hist}} X_{hist,i} \quad \mu_{new} = \frac{1}{n_{new}} \sum_{i=1}^{n_{new}} X_{new,i} \quad (4)$$

(ii) ADWIN then assesses the absolute difference between the means of historical and the new data as: $\Delta = |\mu_{new} - \mu_{hist}|$. (iii) To determine if this difference is statistically significant, ADWIN computes a threshold τ_a using a confidence parameter δ (e.g., $\delta = 0.05$):

$$\tau_a = \sqrt{\frac{1}{2} \ln \left(\frac{4}{\delta} \right) \left(\frac{1}{n_{hist}} + \frac{1}{n_{new}} \right)} \quad (5)$$

(iv) If the difference exceeds the threshold $\Delta > \tau_a$, then we conclude that a significant drift has occurred between X_{hist} and X_{new} .

Autoencoder-Based Approach: In the EDDD approach, we also use an autoencoder, which is an application of deep learning, and its architecture is shown in Fig. 4. The autoencoder is trained on X_{hist} and is used to evaluate the reconstruction errors of X_{new} . The autoencoder computes the reconstruction error for the new data as formulated in Eq. (6).

$$E_{recon} = \frac{1}{n} \sum_{i=1}^n |X_{new}^{(i)} - \hat{X}_{new}^{(i)}| \quad (6)$$

where $\hat{X}_{new}^{(i)}$ is the reconstructed version of sample i . Then, concept drift is detected when the reconstruction error E_{recon} exceeds a dynamically determined threshold τ_b , formulated as:

$$\mathbb{1}(E_{recon}(t) - \tau_b) = \begin{cases} 1, & \text{if } E_{recon}(t) > \tau_b \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

where τ_b is an empirically determined threshold based on prior experimentation and statistical analysis.

Decision Mechanism in EDDD: The ensemble decision function E is defined as a logical OR operation between the drift detection mechanisms of ADWIN and *Autoencoder* (AE). Drift is detected if either

Traditional Network Dataset										
	ts	id.orig_h	duration	orig_bytes	resp_bytes	orig_pkts	orig_ip_bytes	resp_pkts	resp_ip_bytes	proto_icmp
0	518258	1352	545	1	1	5	80	3	48	0
1	518257	1352	2579	13	170	1	9	1	108	0
2	518256	1352	43119	379	1	48	1049	0	0	0

IoT Network Dataset								
	Protocol	Flow Duration	Fwd Packets Length Total	Bwd Packets Length Total	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	Bwd Packet Length Max
0	2	47366	1227	0	341	225	7706	0
1	2	1	444	0	349	301	8034	0
2	2	47	425	0	331	283	7931	0

Transformed M-En Network Dataset								
	0	1	2	3	4	5	6	7
0	5.516609	3.980228	2.672321	-2.975865	1.323544	-0.241056	-0.194046	-2.026733
1	7.281296	3.897008	0.307558	-2.073243	-0.193115	0.041484	0.081345	-1.595307
2	13.443693	-1.404089	1.085177	8.634282	-2.093667	2.517438	-2.169131	6.619309

Fig. 3. M-En Dataset Generation Approach.

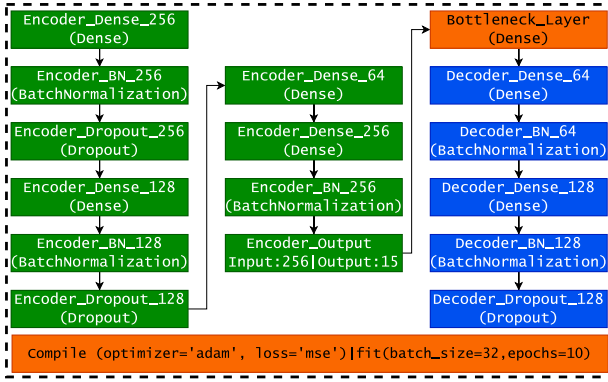


Fig. 4. Autoencoder Architecture For Drift Detection.

ADWIN signals a change or if the reconstruction loss of the AE exceeds a predefined threshold.

$$D_{ADWIN}(t) = \mathbb{1}(\text{ADWIN}(X_{\text{new}}) > \tau_a) \quad (8)$$

$$D_{AE}(t) = \mathbb{1}(\mathcal{L}_{AE}(X_{\text{new}}) > \tau_b) \quad (9)$$

where: (i) t is time step; (ii) $D_{ADWIN}(t)$ is the binary drift detection output of ADWIN; (iii) $D_{AE}(t)$ is the binary drift detection output of the AE; (iv) $\mathbb{1}(\cdot)$ is the indicator function that returns 1 if the condition is true and 0 otherwise; (v) $\text{ADWIN}(X_t)$ represents the ADWIN test statistic computed over the incoming data window; and (vi) $\mathcal{L}_{AE}(X_t)$ is the reconstruction loss of the AE for the given data. Since drift is detected if either method triggers, we define the ensemble decision function as:

$$E(t) = \begin{cases} 1, & \text{if } D_{ADWIN}(t) = 1 \text{ or } D_{AE}(t) = 1 \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

This EDDD ensemble architecture ensures that the system remains robust to gradual distribution changes while still being sensitive to sudden drifts.

3.3. Incremental learning methodology

Our approach combines drift detection with incremental learning using a DNN model [45]. Upon detecting drift, new data is used to fine-tune the model, enabling adaptation to distribution shifts while preserving prior knowledge [46]. This section covers model architecture and incremental learning.

DNN Architecture: The model is implemented using multiple dense layers and dropout for regularization. The architecture is described as follows: The input layer of the model takes in a feature space $X \in \mathbb{R}^{15}$, representing 15 features such as network traffic characteristics like packet size and protocol type. This input layer is followed by the first hidden layer, which is a dense layer with 128 units and ReLU activation [47]. Mathematically, the transformation is defined as $h_1 = \text{ReLU}(W_1 X + b_1)$, where $W_1 \in \mathbb{R}^{128 \times 15}$ and $b_1 \in \mathbb{R}^{128}$. ReLU is used for its non-linearity and to avoid vanishing gradients, enabling effective learning of complex patterns [48]. The second hidden layer is another dense layer with 128 units and ReLU activation, defined as $h_2 = \text{ReLU}(W_2 h_1 + b_2)$, where $W_2 \in \mathbb{R}^{128 \times 128}$ and $b_2 \in \mathbb{R}^{128}$. This layer further extracts and transforms features, enhancing the model's ability to capture abstract patterns essential for classification tasks.

To prevent overfitting, a dropout layer with a rate of 0.5 is applied, where $h_3 = \text{Dropout}(h_2, \text{rate} = 0.5)$. Dropout randomly sets 50% of the units to zero during training, ensuring the model does not overly rely on specific neurons and learns more generalized patterns [49]. Next, the third hidden layer is introduced with 64 units and ReLU activation: $h_4 = \text{ReLU}(W_3 h_3 + b_3)$, where $W_3 \in \mathbb{R}^{64 \times 128}$ and $b_3 \in \mathbb{R}^{64}$. This layer reduces the dimensionality of the feature space, focusing on the most relevant features and improving the model's efficiency. Following this, another dropout layer with a rate of 0.5 is applied $h_5 = \text{Dropout}(h_4, \text{rate} = 0.5)$, reinforcing the model's generalization capabilities. The fourth hidden layer further condenses the features into 32 units with ReLU activation: $h_6 = \text{ReLU}(W_4 h_5 + b_4)$, where $W_4 \in \mathbb{R}^{32 \times 64}$ and $b_4 \in \mathbb{R}^{32}$. As the model becomes deeper, fewer units are necessary to distill the feature space, retaining only the most critical information relevant to the classification task. Finally, the output layer applies a softmax activation for binary classification: $\hat{y} = \text{Softmax}(W_5 h_6 + b_5)$, where $W_5 \in \mathbb{R}^{2 \times 32}$ and $b_5 \in \mathbb{R}^2$. Softmax is used to produce a probability distribution over the classes, enabling the model to predict class probabilities and make decisions based on the likelihood of each class [50].

The model is compiled with the Adam optimizer, binary cross-entropy loss, and accuracy as the evaluation metric. The architecture of the used DNN is illustrated in Fig. 5.

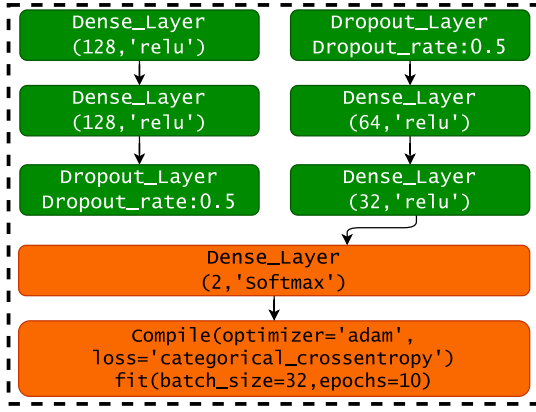


Fig. 5. DNN Architecture For Attack Detection.

Incremental Learning using DNN: The incremental learning function fine-tunes the DNN model when new batches of data arrive, ensuring it remains adaptable while preventing unnecessary model updates. The key steps include tracking weight changes and measuring performance metrics. Let, W_{init} the initial model weights and W_{new} the updated model weights after fine-tuning. The function first retrieves the initial model weights W_{init} and then performs incremental training on the new batch X_{batch}, Y_{batch} :

$$W_{new} = \text{fit}(X_{batch}, Y_{batch}, \text{epochs} = 10, \text{batch_size} = 32) \quad (11)$$

To measure the change in model weights, we calculate the absolute difference as formulated in Eq. (12). After a drift is detected by EDDD, the model is retrained incrementally on the new data, and the change in model weights is recorded each time. This helps identify whether any drastic changes have occurred in the model.

$$\Delta W = \sum_{i=1}^n |W_{new}^{(i)} - W_{init}^{(i)}| \quad (12)$$

The total weight change ΔW represents the overall model adaptation. Additionally, the percentage change is computed for each weight layer:

$$\text{Percentage Change} = \frac{\Delta W}{|W_{init}| + 1e-7} \times 100 \quad (13)$$

Further, to address the catastrophic forgetting problem in incremental learning [51], we use the replay method, which involves storing a subset of past data and mixing it with new data during training to help the model retain prior knowledge. We also use the regularization method, which introduces a penalty term to the loss function, such as $\lambda \|W - W_{old}\|^2$, to prevent large deviations from previously learned weights. Additionally, we apply weight interpolation [52], which updates the model parameters by combining old and new weights using a factor α , $W_{new} = \alpha W_{old} + (1 - \alpha) W_{update}$.

4. Experimentation setup

Data Splitting: or data balancing, we extract 50,000 samples each from the IoT23 and CICDDoS2019 datasets. For initial training purposes, the dataset is divided into a training set (80%) and a testing set (20%), ensuring a robust evaluation of the model's performance. The dataset is structured to represent two classes: Attack (1) and Benign (0). Table 2 presents the distribution of samples in the M-En dataset.

Machine Learning Algorithms: We deploy SOTA machine learning models to compare their performance with our proposed incremental learning approach. These models include RF [53], GBM [54], ADA [54], SVC [55], LR [56], KNN [57], and SGD classifiers [58]. Each model

Table 2

M-En dataset sample count.

Dataset	Total	Benign	Attack
IoT23	50,000	23,215	26,785
CICDDoS2019	50,000	7,253	42,747
M-En	100,000	30,468	69,532

Table 3

Hyperparameters of Deployed SOTA Models.

Model	Hyperparameters	Tuning range
RF	n_estimators = 200 max_depth = 20	n_estimators = [20 to 300] max_depth = [2 to 30]
GBM	n_estimators = 200 learning_rate = 0.05 max_depth = 20	n_estimators = [20 to 300] learning_rate = [0.01 to 0.1] max_depth = [2 to 30]
ADA	n_estimators = 200 learning_rate = 0.05	n_estimators = [20 to 300] learning_rate = [0.01 to 0.1]
SVC	C = 1 kernel = rbf gamma = scale	C = [1 to 5] kernel = [rbf, linear]
LR	Solver = liblinear C = 3.0	[liblinear, saga] C = [1.0 to 10.0]
KNN	n_neighbors = 5 weights = uniform	n_neighbors = [3 to 15]
SGDC	max_iter = 100 warm_start = True loss = log_loss	max_iter = [10 to 1500]

is trained using optimal hyperparameters, determined through a combination of literature review and grid search optimization to enhance generalization and efficiency [59,59–61]. The RF model is configured with 200 estimators, which helps improve ensemble stability, while a maximum depth of 20 balances computational efficiency with capturing complex feature interactions [6]. The GBM and ADA models are both trained with 200 estimators to prevent overfitting, and a learning rate of 0.05 is chosen to optimize convergence speed and model performance [62]. The SVC employs an RBF kernel, which is more effective than a linear kernel for non-linearly separable data [63], while the regularization parameter is set to $C = 1$ to control the balance between maximizing the margin and minimizing classification errors [64]. The kernel coefficient is set to 'scale', allowing automatic adjustment based on input feature variance. For KNN, we set the number of neighbors to 5, striking a balance between bias and variance, while using uniform weights to treat all neighbors equally [65]. LR is used as a baseline model and is trained using the Liblinear solver, which is optimized for binary classification and supports L1 and L2 regularization [66]. Lastly, the SGDC is designed for incremental learning, making it suitable for evolving network traffic patterns [67]. It uses the log-loss function for probabilistic binary classification and is trained with a maximum of 100 iterations ($\text{max_iter} = 100$) to ensure computational efficiency. The warm start feature is enabled, allowing the model to retain knowledge across multiple training iterations, which is crucial for online learning scenarios. SOTA algorithm's specific configurations are summarized in Table 3.

Evaluation Matrices: The performance of proposed approach is measured using several metrics include recall ($\text{Rec} = \frac{TP}{TP+FN}$), precision ($\text{Pre} = \frac{TP}{TP+FP}$), F1-score ($\text{F1} = \frac{2 \cdot \text{Pre} \cdot \text{Rec}}{\text{Pre} + \text{Rec}}$), and accuracy ($\text{Acc} = \frac{TP+TN}{TP+TN+FP+FN}$), alongside computational aspects such as processing time, model size, and memory consumption. Here, (i) *True Positive* (TP) is the number of attack instances correctly classified as attacks. (ii) *True Negative* (TN) is the number of normal instances correctly classified as normal. (iii) *False Positive* (FP) is the number of normal instances incorrectly classified as attacks (also called false alarms). (iv) *False Negative* (FN) is the number of attack instances incorrectly classified as normal (missed detections).

XAI Implementation: We used LIME to interpret individual predictions of our classifier. The explainer was initialized with the training

data, set to classification mode, and provided with feature and class names to improve interpretability. A test instance was selected, and `explain_instance`³ was applied using the model's probability predictions and the complete feature set to generate a detailed explanation. This setup allowed us to identify which features most influenced the model's decision for that specific instance.

5. Results & discussion

For the results, all experiments in this study were conducted on a 12th-generation Intel Core i7 machine running Windows, equipped with 64 GB of RAM and a 1TB SSD.

5.1. Results using proposed framework: EDDD and incremental learning

This section discusses the results of our approach using EDDD for drift detection and incremental learning. A dataset of 100,000 samples was processed in batches of 10,000. The first batch was used to train and evaluate the DNN model without drift detection. The autoencoder was also trained on this initial batch to enable drift detection for the following batches. For each subsequent 10,000 sample batch, the EDDD method checked for drift. If drift was detected, the model and autoencoder were incrementally updated; otherwise, only the autoencoder was retrained. After each batch, we evaluated the model's performance. In the first batch, the model achieved 0.96 Acc, with an *model size* (MS) of 11 MB and 30.91 MB of *memory use* (MU). In the subsequent batches, even when handling class imbalance during incremental learning, we observed a consistent improvement in accuracy, reaching 0.99 to 1.00 by the final batch. Despite significant changes in model weights due to incremental learning, the MS and MU remained stable, as shown in Table 4. This is because the model architecture, including the number of layers and parameters, was not altered during the incremental learning process. Consistent drift detection across batches demonstrates EDDD's effectiveness, unlike other methods that may miss drift and risk performance degradation (see Section 5.2).

We also deployed the proposed framework to specifically address the known risk of model bias toward new data during incremental fine-tuning by incorporating two key strategies: data replay and implicit regularization. To mitigate catastrophic forgetting, we applied a replay mechanism in which 10% of previously seen data is combined with new incoming data in each batch. This ensures the model retains knowledge of earlier distributions while adapting to concept drift. Additionally, the model demonstrates stable learning behavior, with small weight changes and consistently low standard deviation in accuracy across multiple seeds, indicating the presence of implicit regularization through controlled training dynamics. Evaluation results confirm that the model maintains high accuracy not only on current batches but also on old data an average of 0.976. To ensure robust evaluation, we retrain the model multiple times per batch using different random seeds. This allows us to compute both the mean and *standard deviation* (SD) of accuracy, reflecting performance stability and mitigating the effects of randomness. We used five seeds, 0, 42, 7, 99, and 123, for these repeated evaluations. Table 5 shows that our approach effectively prevents overfitting to new data while preserving prior knowledge and improving accuracy over time with incremental learning.

Furthermore, we introduced an interpolation-based weight update strategy governed by a blending parameter $\alpha \in [0, 1]$, as defined in Eq. (2). This parameter controls the trade-off between retaining historical knowledge and incorporating new information. We evaluated the model using α values of 0.0, 0.25, 0.5, 0.75, and 1.0. The results showed that intermediate values, particularly between 0.25 and 0.75, provided the best balance by achieving high accuracy up to 0.999 and maintaining low standard deviation across batches. In contrast,

setting $\alpha = 1.0$, where the model fully relies on previously learned weights without incorporating new data, resulted in severe forgetting, with accuracy dropping below 50% in later batches. On the other end, $\alpha = 0.0$, which replaces prior knowledge entirely with new data, also led to instability. These results indicate that balanced values of α (i.e., 0.25, 0.5, and 0.75) effectively blend past and present knowledge, enabling the model to remain robust and adaptable in the presence of concept drift as shown in Fig. 6.

We conducted an additional experiment to demonstrate the effectiveness of our proposed approach. To do so, we applied a test-then-train strategy, which allows us to highlight the critical importance of accurately and efficiently detecting data drift using EDDD. Fig. 7 illustrates that the pre-retraining accuracy was significantly low in several cases. However, after applying incremental retraining, the model's performance improved substantially. For instance, in Batch 6, a sudden and drastic shift in data distribution led to an accuracy close to zero, but after retraining, the model achieved near-perfect performance. Similarly, in Batches 7 and 8, substantial changes caused poor performance before retraining, which was effectively improved afterward. The results show that without drift detection and incremental adaptation, performance degrades, while our EDDD approach maintains accuracy by detecting and adjusting to both minor and major drifts.

To validate that our proposed approach does not rely on time-based features, we removed the *Timestamp* (TS) attribute from the IoT23 dataset. This modification we made to eliminate any potential temporal bias and ensure that the model learns solely from intrinsic traffic characteristics. Following this change, the initial evaluation showed an accuracy of 0.94 in the first batch. As the model incrementally adapted to new traffic patterns in subsequent batches, its performance improved progressively, ultimately achieving near-perfect results as shown in Fig. 8. The robustness and generalizability of our approach without TS features confirm that it effectively captures relevant behavioral features without depending on timing information.

5.2. Performance comparison of drift detection methods

To assess the effectiveness of different drift detection mechanisms, we compared Autoencoder-based detection, ADWIN, and our proposed EDDD. Detailed per-batch results have been relocated to Table A.8 and Table A.9. The Autoencoder approach achieved consistent performance across batches, with accuracy stabilizing between 0.94 and 0.96 and reaching 1.00 by batch 6. It demonstrated adaptability to changing data distributions, though it failed to detect drift in batch 7. Similarly, ADWIN maintained high accuracy, peaking at 1.00 after batch 6. However, it missed drift in batch 9, leading to skipped retraining and no performance updates. Both methods showed significant weight adjustments in early batches, reflecting model adaptation, while memory usage and model size remained stable with 22.66 MB and 0.11 MB, respectively. Fig. 9 compares these methods with EDDD. The EDDD approach detects drift more reliably across batches, resulting in better model stability and reduced weight fluctuations, indicating superior adaptability and learning efficiency in dynamic environments. The EDDD mechanism continuously detects changes by combining the strengths of both statistical methods and a neural network-based encoder.

5.3. Comparison with other studies

In this section, we compare our study with several others in terms of accuracy and the use of techniques such as XAI, IL, and drift detection (DD). For a fair comparison, all approaches were deployed on our M-En dataset using the same experimental setup. The selected studies either focused on handling heterogeneous datasets or adopted IL approaches. For instance, the study by [7] employed an ensemble

³ <https://github.com/marcotcr/lime>

Table 4
Results using proposed framework.

Batch	Classification report					CM	
	Acc	Class	Pre	Rec	F1	Value	Count
1	0.96	0	0.93	1.00	0.96	TP	893
		1	1.00	0.93	0.97	FP	0
		MA	0.96	0.97	0.96	TN	1035
		WA	0.97	0.96	0.96	FN	72
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	–	–	28 962	–	–	0.11	30.91
2	0.94	0	0.91	1.00	0.95	TP	1097
		1	1.00	0.88	0.93	FP	0
		Mi. Avg	0.95	0.94	0.94	TN	791
		Wt. Avg	0.95	0.94	0.94	FN	112
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	404.00	144.25%	0.11	30.91
3	0.96	0	0.93	1.00	0.97	TP	1140
		1	1.00	0.90	0.95	FP	0
		MA	0.97	0.95	0.96	TN	778
		WA	0.96	0.96	0.96	FN	82
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	465.00	100.85%	0.11	30.91
4	0.95	0	0.93	1.00	0.96	TP	1128
		1	1.00	0.90	0.94	FP	0
		MA	0.96	0.95	0.95	TN	781
		WA	0.96	0.95	0.95	FN	91
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	350.04	104.49%	0.11	30.91
5	0.96	0	0.94	1.00	0.97	TP	1159
		1	1.00	0.91	0.95	FP	0
		MA	0.97	0.96	0.96	TN	768
		WA	0.97	0.96	0.96	FN	73
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	535.32	114.94%	0.11	30.91
6	1.00	0	1.00	1.00	1.00	TP	2000
		1	–	–	–	FP	–
		MA	1.00	1.00	1.00	TN	–
		WA	1.00	1.00	1.00	FN	–
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	752.93	113.39%	0.11	30.91
7	0.99	0	1.00	0.99	1.00	TP	1128
		1	0.99	1.00	0.99	FP	9
		MA	0.99	0.99	0.99	TN	870
		WA	0.99	0.99	0.99	FN	2
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	1320.33	214.56%	0.11	30.91
8	1.00	0	1.00	1.00	1.00	TP	1437
		1	0.99	1.00	0.99	FP	6
		MA	0.99	1.00	1.00	TN	556
		WA	1.00	1.00	1.00	FN	1
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	813.48	113.44%	0.11	30.91
9	1.00	0	1.00	1.00	1.00	TP	2000
		1	–	–	–	FP	–
		MA	1.00	1.00	1.00	TN	–
		WA	1.00	1.00	1.00	FN	–
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	172.24	36.16%	0.11	30.91
10	1.00	0	1.00	1.00	1.00	TP	1999
		1	–	–	–	FP	1
		MA	0.50	0.50	0.50	TN	–
		WA	1.00	1.00	1.00	FN	–
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	49.19	8.61%	0.11	30.91

Acronyms: Accuracy (Acc); Precision (Pre); Recall (Rec); F1 Score (F1); Weights Changed (WC); Total Weights (TW); Total Change (TC); Average % Change (AC); Model Size (MS); Memory Usage (MU); Weighted Average (WA); Macro Average (MA); Confusion Matrix (CM).

Table 5
Proposed framework results with replay, regularization, and 5 seeds.

Batch	Mean Acc	SD	Batch	Mean Acc	SD
1	0.9685	± 0.0075	6	0.9998	± 0.0002
2	0.9502	± 0.0043	7	0.9939	± 0.0022
3	0.9602	± 0.0032	8	0.9948	± 0.0014
4	0.9583	± 0.0032	9	0.9992	± 0.0002
5	0.9645	± 0.0036	10	0.9979	± 0.0006

method for detecting cyberattacks in M-En, focusing primarily on enhancing accuracy without incorporating DD, IL, or XAI. Similarly, the study by [68] proposed an *Adaptive Random Forest Classifier* (ARFC), which incorporated IL to continually update the model with new data. Another study by [6] introduced an MFO-based approach optimized for cyberattack detection. Likewise, [2] presented a PSO-based (Particle Swarm Optimization) self-ensemble method tailored for attack detection on the M-En dataset. In contrast, the study by [69] proposed an *accuracy-update weighted probability averaging ensemble* (AUWPAE) that incorporated both drift detection and incremental learning. Although

their model was not originally tested on the M-En dataset, we adapted it for comparison. Further, we also considered error-rate-based drift detection techniques in combination with incremental learning for comparison with our approach, such as [26], which deploys an LSTM-based approach. However, the approach is limited in detecting attacks in M-En settings and has comparatively restricted drift detection capabilities compared to our approach. All models were deployed on the M-En dataset, and the comparative results are presented in Table 6. Our proposed approach, which combines the EDDD method with a DNN, stands out by integrating XAI, DD, and IL. When tested on the M-En dataset,

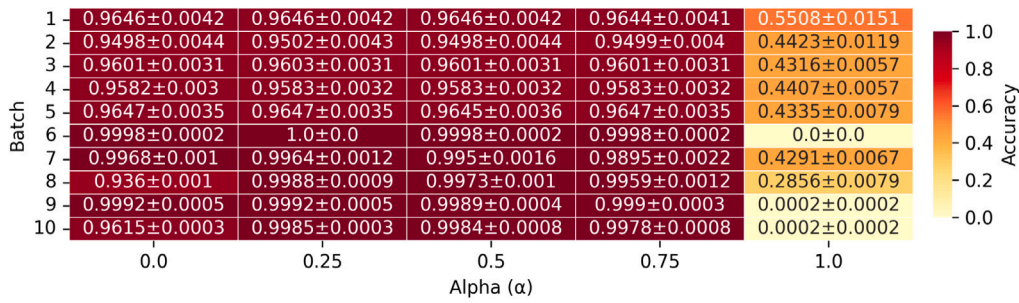
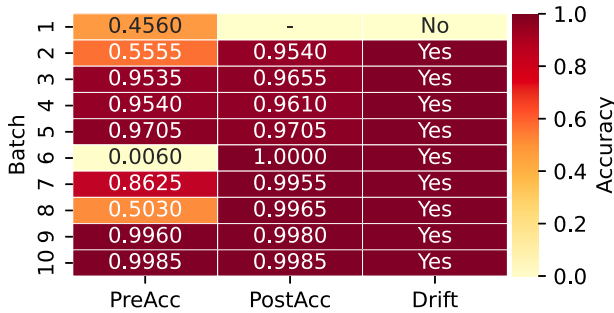
Fig. 6. Proposed Framework Accuracy Across Batches for Varying α Values.

Table 6

Performance comparison with other studies.

Ref	Year	Method	M-En	Acc	XAI	DD	IL
[7]	2021	Ensemble Method	✓	0.960	X	X	X
[68]	2022	ARFC	X	0.973	X	X	✓
[6]	2024	MFO-based Optimized Approach	✓	0.981	X	X	X
[2]	2024	PSO-based Self-Ensemble	✓	0.985	X	X	X
[69]	2024	AUWPAE	X	0.941	X	✓	✓
[26]	2025	CDDA-MD	X	0.967	X	✓	✓
Our	2025	EDDD + DNN	✓	0.999	✓	✓	✓

Fig. 7. Impact of EDDD on Model Performance; *PreAcc* – Accuracy before retraining; *PostAcc* – Accuracy after retraining.

our method achieved a top accuracy of 0.999, offering a comprehensive solution that not only adapts to data distribution changes but also provides interpretable insights into the model's decision-making process.

5.4. Discussion

This study presents an adaptive framework for malicious traffic detection in M-En networks. The proposed approach demonstrates strong performance under the EDDD mechanism combined with incremental training when new traffic patterns emerge. We evaluated the framework across various incremental learning scenarios, as illustrated in Fig. 10. Results show that when catastrophic forgetting is actively controlled, using strategies such as replay, regularization, or interpolation, the incremental learning model maintains high accuracy over time. In contrast, models without incremental learning perform poorly, failing to retain useful knowledge. Furthermore, using default incremental learning without any forgetting or retention control leads to inconsistent results.

Our approach is not specifically designed for only two datasets or their associated attacks. It is built to handle any real-time dataset with varying traffic patterns and attack types. To validate this, we included a third, more diverse dataset, EdgeIoTset [70], which contains 14 different attack types such as DDoS, SQL Injection, and Password attacks. Introducing this new dataset helps assess the robustness of our

approach in more complex scenarios. Fig. 11 shows that our method successfully adapts to this new, complex environment with consistently high accuracy across batches, even when we evaluated the model on both new and old data.

Our study not only proposed an adaptive EDDD-based framework for incremental learning but also compared it against several SOTA models in terms of both performance and efficiency. First, we evaluate them with simple train-test methods, and these models achieved high static accuracy up to 0.98 as shown in Table 7. Then, we evaluated them using the incremental process without applying incremental training, as shown in Fig. 12. However, they are not designed for dynamic environments and lack mechanisms for continual learning, making them less suitable for evolving M-En networks. Additionally, SOTA models such as GBM and SVC require significantly longer training times (e.g., 582 s and 266s) and consume more memory, as shown in Table 7, which limits their practicality for real-time deployment. Without incremental learning, these models failed completely, as their performance gradually declined over time. The mean accuracy across 10 batches shows that not a single model surpassed an accuracy score of 0.8. Most models exhibited a significant drop in accuracy or unstable performance in the later batches due to the emergence of new patterns in the incoming traffic, as illustrated in Fig. 12.

Furthermore, regarding real-time deployment and latency, our approach is significant because it includes a feature selection step using the ET method, which took 59.1 s for traditional data and 99.9 s for IoT data. However, this selection is performed only once during the offline training phase. During real-time deployment, only the pre-selected features are used, significantly reducing computational overhead. For feature decomposition using PLS, the initial fitting process took 1.9 s, which includes the fit function. In real-time inference, only the transform function is applied, which is lightweight and introduces no noticeable delay. Therefore, the proposed feature processing pipeline is well-suited for real-time use. Although our system integrates drift detection, incremental learning, and explainability, each component is designed to be lightweight. Modules like the autoencoder-based drift detector and LIME-based explainability are triggered only when needed, minimizing ongoing resource usage. Further in our proposed approach, the lightweight DNN architecture demonstrates strong efficiency, with raw features, the DNN model achieved a mean prediction time of 0.6375 s (± 0.16 s) per 10,000 samples, allowing it to process approximately 15,690 samples/second. Incremental training took 14.67 s

Metric	Accuracy	0.964	0.940	0.960	0.950	0.960	1.000	1.000	1.000	1.000	1.000
	F1	0.960	0.940	0.960	0.950	0.960	1.000	1.000	1.000	1.000	1.000
	Precision	0.960	0.950	0.970	0.960	0.970	1.000	1.000	0.990	1.000	1.000
	Recall	0.970	0.940	0.950	0.950	0.960	1.000	1.000	1.000	1.000	1.000
		1	2	3	4	5	6	7	8	9	10
		Batch Number									

Fig. 8. Proposed Framework Results without TS Feature.

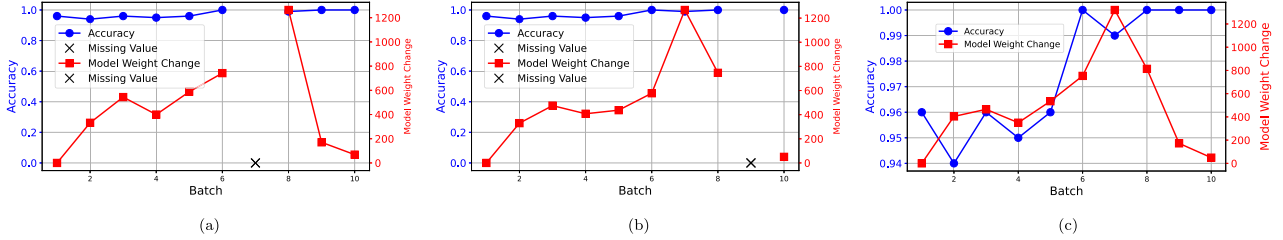


Fig. 9. Accuracy vs Weight Changes: (a) Autoencoder, (b) ADWIN, (c) EDDD.

Table 7

Performance and resource metrics for SOTA models.

Model	Acc	Pre	Rec	F1	TP	FN	FP	TN	TrT	TeT	MU	MS
RF	0.98	0.98	0.96	0.97	13 898	13	442	5647	58.75	0.23	0.001	8.04
GBM	0.97	0.97	0.96	0.96	13 743	168	443	5646	582.48	0.20	27.13	24.01
ADA	0.96	0.96	0.94	0.95	13 742	169	630	5459	69.07	0.43	1.52	0.16
SVC	0.98	0.98	0.96	0.97	13 888	23	450	5639	266.65	7.92	15.93	0.55
LR	0.95	0.96	0.92	0.94	13 797	114	906	5183	0.52	0.0035	0.001	0.003
KNN	0.98	0.98	0.96	0.97	13 881	30	448	5641	0.30	1.50	0.48	20.60
SGDC	0.95	0.96	0.93	0.94	13 798	113	825	5264	0.0751	0.0032	0.001	0.008

TP—True Positive; FN—False Negative; FP—False Positive; TN— True Negative; TrT— Training Time (Sec); TeT— Testing Time (Sec); MU— Memory Use (MB); MS— Model Size (MB);

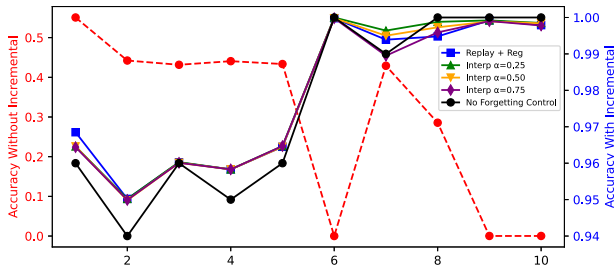


Fig. 10. Accuracy under Different Incremental Learning Strategies.

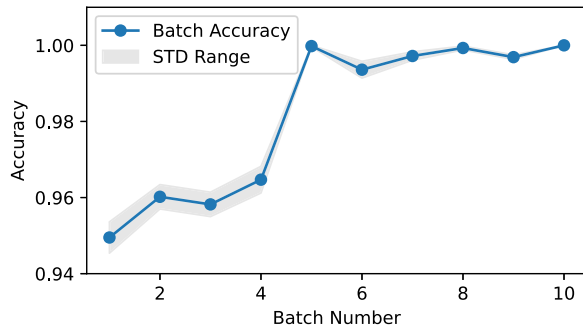


Fig. 11. Proposed Approach Performance with Three Datasets.

($\pm 1.98s$), equating to roughly 681 samples/second. After applying our selected features, the prediction time reduced to 0.527 s ($\pm 0.06s$), resulting in a throughput of about 18,970 samples/second. Similarly, incremental training improved to 11.27 s ($\pm 0.91s$), allowing for approximately 887 samples/second. These results demonstrate that feature selection not only enhances accuracy but also significantly improves computational efficiency, making the model well-suited for real-time, large-scale network monitoring and adaptive threat detection.

5.5. Comparative XAI analysis

We employ LIME to interpret our MTDS [71], initialized with the training data, feature names, and class labels. LIME perturbs a selected instance, observes the model's responses, and fits a local surrogate model to highlight key features driving the prediction.

XAI for Proposed Framework: Fig. 13 illustrates how the DNN confidently classified a specific instance as “Attack” with 100 percent certainty. This decision was mainly driven by features such as Feature 9, with a value greater than 0.12, and Feature 10, with a value greater than 0.22, which strongly pushed the prediction toward the attack class. In contrast, features like Feature 12, with a value greater than -0.07 , and Feature 14, with a value less than -0.05 , offered slight support to the benign class but had minimal influence.

After transforming the input features for modeling, the data becomes abstract, with generic names like *Feature 1*, *Feature 2*, and so on. While this abstraction supports computational efficiency, it makes it difficult to interpret which original features actually influence the model's decisions. To address this, we performed a reverse transformation to map each abstract feature back to its original representation. This allowed us to identify the specific features from each source dataset that contribute most to the predictions. For instance, features such as *conn_state_SH*, *orig_pkts*, and *resp_ip_bytes* were found to be most significant from the IoT dataset, while *Fwd Packets Length Total*,

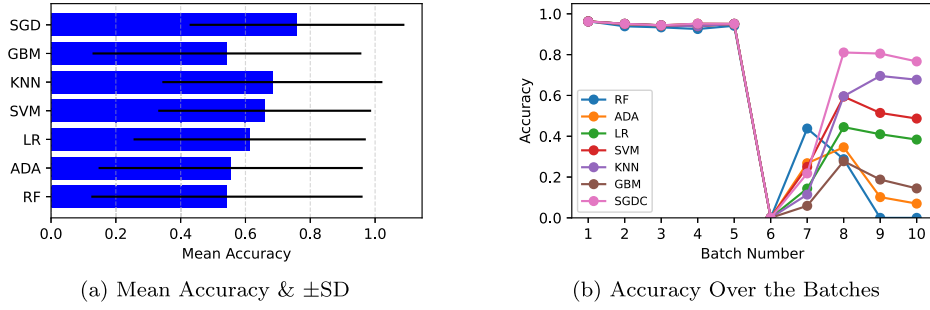


Fig. 12. SOTA Models without Incremental Learning across Sequential Batches.

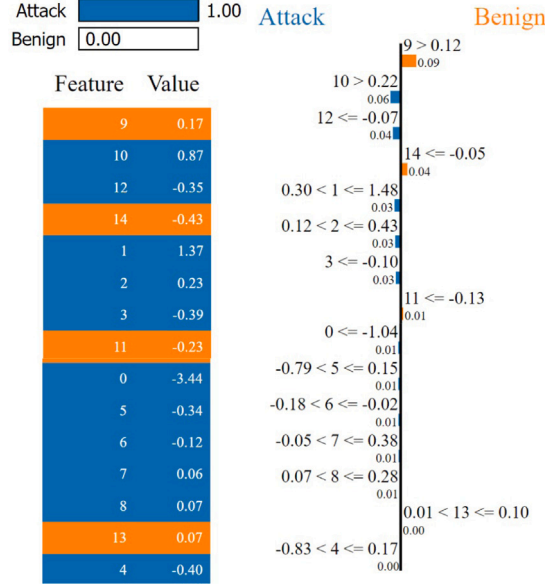


Fig. 13. LIME Visualization For Proposed Framework.

Subflow Fwd Bytes, and Bwd IAT Min emerged as top contributors from the traditional network dataset. The importance of these features is visualized in Fig. 14.

XAI for SOTA RF: Fig. 15(a) shows how the RF model, one of the state-of-the-art baselines, classifies a sample with 0.98 confidence as Class 0. This decision is strongly influenced by features like *proto_tcp*, *proto_udp*, and *orig_ip_bytes*, which contribute positively to the attack prediction. The center panel in the figure illustrates how each feature condition, for example, $proto_tcp \leq 1.00$, pushes the decision toward either Class 0 or Class 1. Alongside this, the table provides the actual values of the features for that specific instance, helping explain how the model arrived at its decision.

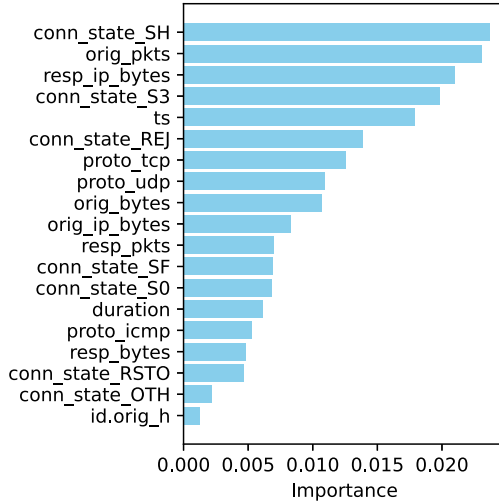
In addition to the LIME visualization, we also used a force plot to illustrate the features influencing the model's decision [72]. Fig. 15(b) shows the force plot for the RF model. The difference between this and the LIME visualization stems from the classes they explain. LIME explains the confident prediction of Class 0 with 0.98 probability, while the force plot illustrates why the instance was not classified as Class 1, starting from a base value of 0.02. Essentially, LIME highlights support for Class 0, whereas the force plot focuses on rejecting Class 1. Key features, such as minimal *res_bytes*, zero-valued *proto_temp*, short duration, and absence of specific connection states (e.g., *conn_state_SH*) contribute negatively for class 1. These feature conditions align with benign traffic patterns, guiding the model's confident rejection of Class 1.

6. Conclusion & future work

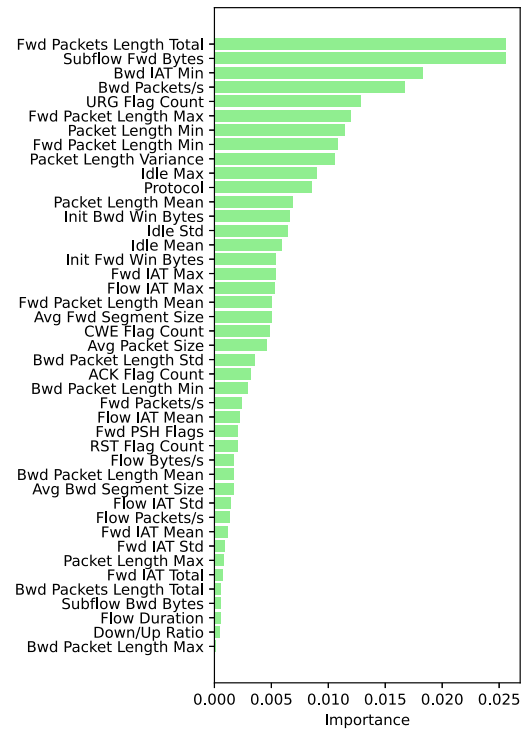
To tackle the heterogeneous nature of M-En networks targeted by malicious actors, this study conducted extensive experimentation

using an adaptive framework. We generated a unified M-En dataset by integrating benchmark datasets and applied an incremental learning approach, which is well-suited for real-time environments, as it updates models with new traffic post-deployment. A data drift detection mechanism was employed to identify previously unseen traffic patterns, enabling the model to retrain only when necessary, thereby preserving computational resources and previously learned knowledge. Our findings indicate that: (i) Drift detection significantly reduces post-deployment computational overhead by avoiding unnecessary model updates. The proposed EDDD approach, which combines both an autoencoder and ADWIN, proved more effective than relying on a single method. It ensures that, whether the change is pattern-based or statistical, at least one detector will trigger retraining. EDDD can detect both minor and drastic drift, helping the model retrain in time to avoid significant degradation. (ii) DNNs are particularly effective in incremental learning settings, achieving up to 0.999 (approximately 1.0) accuracy while maintaining computational efficiency, with a throughput of approximately 8,970 samples per second. (iii) Weight interpolation plays a critical role in mitigating catastrophic forgetting by providing fine-grained control over knowledge retention and adaptation. In contrast, models lacking incremental learning capabilities quickly forget prior knowledge, resulting in degraded performance over time. (iv) XAI analysis revealed that features such as Fwd Packets Length Total, Subflow Fwd Bytes, Bwd IAT Min, Bwd Packets/s, URG Flag Count, *conn_state_SH*, *orig_pkts*, and *resp_ip_bytes* are key indicators for detecting attacks across M-En networks.

Limitations & Future Directions: Despite these significant contributions, there are limitations that we will address in future work. The M-En dataset we generated using benchmark datasets may not

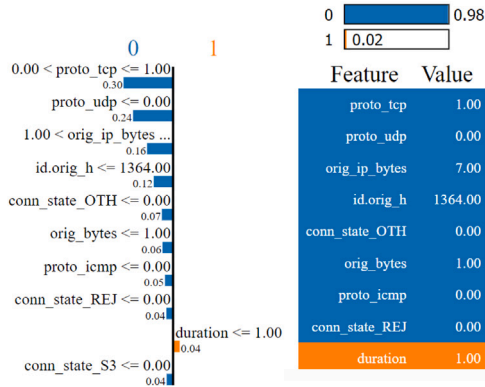


(a) IoT network features

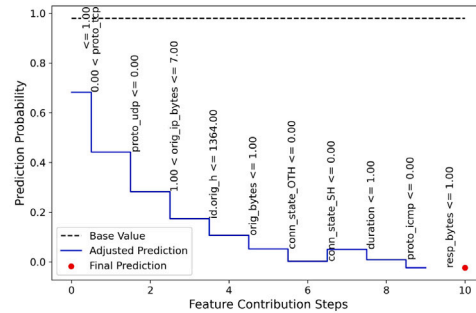


(b) Traditional network features

Fig. 14. Inverse Mapping of M-En Feature Importance: (a) IoT23, (b) CICDDoS2019.



(a) LIME Visualization For the RF Model



(b) Force Plot Visualization For the RF

Fig. 15. SOTA Model XAI Visualization.

fully represent real traffic in M-En networks. To address this, we are currently developing an M-En simulation using the NS-3 tool to collect real traffic data. Secondly, while we proposed our approach for real-time scenarios, we have not yet tested it in real-world networks and have only validated it offline. Our ongoing work involves building a simulation environment for dataset collection and real-world testing. Another limitation of our current approach is its reliance on human experts for labeling data during concept drift, which can hinder real-time responsiveness. To address this, we are developing an LLM-based framework to automate labeling and reduce manual intervention.

CRedit authorship contribution statement

Furqan Rustam: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Anca Delia Jurcut:** Writing – review & editing, Supervision, Resources, Project administration, Investigation, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

All authors have contributed significantly to the work and agree with the content of the manuscript. This publication has emanated from research conducted under the TROCI project, with the financial support of Taighde Éireann – Research Ireland under Grant No. CHIST-ERA-22-SPiDDS-07.

Appendix. Extended evaluation of drift detection approaches

See [Tables A.8](#) and [A.9](#).

Table A.8
Results using autoencoder drift detection and incremental learning.

Batch	Classification report					CM	
	Acc	Class	Pre	Rec	F1	Value	Count
1	0.96	0	0.93	1.00	0.96	TP	893
		1	1.00	0.93	0.97	FP	0
		MA	0.96	0.96	0.96	TN	1035
		WA	0.97	0.96	0.96	FN	72
	Statistics						
	DD	WC	TW	TC	AC	MS	MU
	False	True	28 962	705.41	57122804.0%	0.11	22.66
	Acc	Class	Pre	Rec	F1	Value	Count
2	0.94	0	0.91	1.00	0.95	TP	1097
		1	1.00	0.88	0.93	FP	0
		MA	0.94	0.94	0.94	TN	791
		WA	0.95	0.94	0.94	FN	112
	Statistics						
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	331.78	92.77%	0.11	22.66
	Acc	Class	Pre	Rec	F1	Value	Count
3	0.96	0	0.93	1.00	0.96	TP	1140
		1	1.00	0.90	0.95	FP	0
		MA	0.96	0.96	0.96	TN	777
		WA	0.96	0.96	0.96	FN	83
	Statistics						
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	544.11	190.25%	0.11	22.66
	Acc	Class	Pre	Rec	F1	Value	Count
4	0.95	0	0.93	1.00	0.96	TP	1128
		1	1.00	0.90	0.94	FP	0
		MA	0.95	0.95	0.95	TN	781
		WA	0.96	0.95	0.95	FN	91
	Statistics						
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	399.63	176.46%	0.11	22.66
	Acc	Class	Pre	Rec	F1	Value	Count
5	0.96	0	0.94	1.00	0.97	TP	1159
		1	1.00	0.91	0.95	FP	0
		MA	0.96	0.96	0.96	TN	768
		WA	0.97	0.96	0.96	FN	73
	Statistics						
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	588.34	133.47%	0.11	22.66

Batch	Classification report					CM	
	Acc	Class	Pre	Rec	F1	Value	Count
6	1.00	0	1.00	1.00	1.00	TP	2000
		1	–	–	–	FP	–
		MA	1.00	1.00	1.00	TN	–
		WA	1.00	1.00	1.00	FN	–
	Statistics						
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	742.12	136.08%	0.11	22.66
	Acc	Class	Pre	Rec	F1-Score	Value	Count
7	–	–	–	–	–	TP	–
		–	–	–	–	FP	–
		MA	–	–	–	TN	–
		WA	–	–	–	FN	–
	Statistics						
	DD	WC	TW	TC	AC	MS	MU
	False	True	–	–	–	–	–
	Acc	Class	Pre	Rec	F1	Value	Count
8	0.99	0	1.00	0.99	0.99	TP	1117
		1	0.99	1.00	0.99	FP	11
		MA	0.99	0.99	0.99	TN	870
		WA	0.99	0.99	0.99	FN	2
	Statistics						
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	1263.92	193.95%	0.11	22.66
	Acc	Class	Pre	Rec	F1	Value	Count
9	1.00	0	1.00	1.00	1.00	TP	1443
		1	0.99	1.00	0.99	FP	6
		MA	1.00	1.00	1.00	TN	557
		WA	1.00	1.00	1.00	FN	0
	Statistics						
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	170.43	127.28%	0.11	22.66
	Acc	Class	Pre	Rec	F1	Value	Count
10	1.00	0	1.00	1.00	1.00	TP	2000
		1	–	–	–	FP	–
		MA	1.00	1.00	1.00	TN	–
		WA	1.00	1.00	1.00	FN	–
	Statistics						
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	68.69	18.88%	0.11	22.66

Table A.9
Results using ADWIN drift detection and incremental learning.

Batch	Classification report					CM	
	Acc	Class	Pre	Rec	F1-Score	Value	Count
1	0.96	0	0.93	1.00	0.96	TP	892
		1	1.00	0.93	0.97	FP	1
		MA	0.96	0.96	0.96	TN	1035
		WA	0.97	0.96	0.96	FN	72
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	–	–	28 962	–	–	0.11	22.66
2	0.94	0	0.91	1.00	0.95	TP	1097
		1	1.00	0.88	0.93	FP	0
		MA	0.94	0.94	0.94	TN	791
		WA	0.95	0.94	0.94	FN	112
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	329.75	137.14%	0.11	22.66
3	0.96	0	0.93	1.00	0.96	TP	1140
		1	1.00	0.90	0.95	FP	0
		MA	0.96	0.96	0.96	TN	777
		WA	0.96	0.96	0.96	FN	83
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	False	True	28 962	474.85	134.77%	0.11	22.66
4	0.95	0	0.93	1.00	0.96	TP	1128
		1	1.00	0.90	0.94	FP	0
		MA	0.95	0.95	0.95	TN	781
		WA	0.96	0.95	0.95	FN	91
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	407.43	77.14%	0.11	22.66
5	0.96	0	0.94	1.00	0.97	TP	1159
		1	1.00	0.91	0.95	FP	0
		MA	0.96	0.96	0.96	TN	786
		WA	0.97	0.96	0.96	FN	73
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	437.09	145.13%	0.11	22.66

Batch	Classification report					CM	
	Acc	Class	Pre	Rec	F1-Score	Value	Count
6	1.00	0	1.00	1.00	1.00	TP	2000
		1	–	–	–	FP	–
		MA	1.00	1.00	1.00	TN	–
		WA	1.00	1.00	1.00	FN	–
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	577.40	205.95%	0.11	22.66
7	0.99	0	1.00	0.99	0.99	TP	1117
		1	0.99	1.00	0.99	FP	11
		MA	0.99	0.99	0.99	TN	870
		WA	0.99	0.99	0.99	FN	2
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	1267.32	170.98%	0.11	22.66
8	1.00	0	1.00	1.00	1.00	TP	1439
		1	0.99	1.00	1.00	FP	4
		MA	1.00	1.00	1.00	TN	557
		WA	1.00	1.00	1.00	FN	0
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	746.57	138.41%	0.11	22.66
9	–	–	–	–	–	TP	–
		–	–	–	–	FP	0
		MA	–	–	–	TN	–
		WA	–	–	–	FN	–
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	False	True	–	–	–	–	–
10	1.00	0	1.00	1.00	1.00	TP	1999
		1	–	–	–	FP	1
		MA	1.00	1.00	1.00	TN	–
		WA	1.00	1.00	1.00	FN	–
		Statistics					
	DD	WC	TW	TC	AC	MS	MU
	True	True	28 962	50.36	8.61%	0.11	22.66

M-En dataset & code

We have made the datasets used to generate the M-En dataset, along with the Python code, available at the given link. https://drive.google.com/drive/folders/1Lo7eivA_7A9-YSQmC-KLQBS0RoLtUP7.

References

- [1] Syunyurenko O, Gilyarevskii R. Trends and risks of network technologies. *Sci Tech Inf Process* 2021;48:97–106.
- [2] Rustam F, Jurcut AD. Malicious traffic detection in multi-environment networks using novel S-DATE and PSO-D-SEM approaches. *Comput Secur* 2024;136:103564.
- [3] Mijumbi R, Serrat J, Gorricho J-L, Bouten N, De Turck F, Boutaba R. Network function virtualization: State-of-the-art and research challenges. *IEEE Commun Surv Tutor*. 2016;18(1):236–62.
- [4] Simeone O. A very brief introduction to machine learning with applications to communication systems. *IEEE Trans Cogn Commun Netw* 2018;4(4):648–64.
- [5] Rustam F, Jurcut AD, Aljedaani W, Ashraf I. Securing multi-environment networks using versatile synthetic data augmentation technique and machine learning algorithms. In: 2023 20th annual international conference on privacy, security and trust. IEEE; 2023, p. 1–10.
- [6] Rustam F, Aljedaani W, Elsayed MS, Jurcut AD. FAMTDS: A novel MFO-based fully automated malicious traffic detection system for multi-environment networks. *Comput Netw* 2024;251:110603.
- [7] Indrasiri PL, Lee E, Rupapara V, Rustam F, Ashraf I. Malicious traffic detection in iot and local networks using stacked ensemble classifier. *Comput Mater Contin* 2022;71(1):489–515.
- [8] Nguyen TG, Phan TV, Hoang DT, Nguyen TN, So-In C. Federated deep reinforcement learning for traffic monitoring in SDN-based IoT networks. *IEEE Trans Cogn Commun Netw* 2021;7(4):1048–65.
- [9] Cherian M, Chatterjee M. Survey of security threats in IoT and emerging countermeasures. In: Thampi SM, Madria S, Wang G, Rawat DB, Alcaraz Calero JM, editors. *Security in computing and communications*. Singapore: Springer Singapore; 2019, p. 591–604.
- [10] Tan SF, Samsudin A. Recent technologies, security countermeasure and on-going challenges of Industrial Internet of Things (IIoT): A survey. *Sensors* 2021;21(19):6647.
- [11] Aamir M, Zaidi MA. Ddos attack and defense: Review of some traditional and current techniques. 2014, arXiv preprint arXiv:1401.6317.
- [12] Pakmehr A, Alsmuth A, Taheri N, Ghaffari A. Ddos attack detection techniques in IoT networks: a survey. *Clust Comput* 2024;1–32.
- [13] Ghorbanian M, Dolatabadi SH, Masjedi M, Siano P. Communication in smart grids: A comprehensive review on the existing and future communication and information infrastructures. *IEEE Syst J* 2019;13(4):4001–14.
- [14] Catarinucci L, de Donno D, Mainetti L, Palano L, Patrono L, Stefanizzi ML, et al. An IoT-aware architecture for smart healthcare systems. *IEEE Internet Things J* 2015;2(6):515–26.

- [15] Huo Y, Liang W, Chen J, Zhuang S, Sun J. LightGuard: A lightweight malicious traffic detection method for internet of things. *IEEE Internet Things J* 2024.
- [16] Liu H, Han F, Zhang Y. Malicious traffic detection for cloud-edge-end networks: A deep learning approach. *Comput Commun* 2024;215:150–6.
- [17] Sayed MSE, Le-Khac N-A, Azer MA, Jurcut AD. A flow-based anomaly detection approach with feature selection method against ddos attacks in SDNs. *IEEE Trans Cogn Commun Netw* 2022;8(4):1862–80.
- [18] Wang S, Gomez K, Sithamparanathan K, Asghar MR, Russello G, Zanna P. Mitigating ddos attacks in sdn-based iot networks leveraging secure control and data plane algorithm. *Appl Sci* 2021;11(3):929.
- [19] Aljuhani A. Machine learning approaches for combating distributed denial of service attacks in modern networking environments. *IEEE Access* 2021;9:42236–64.
- [20] Indusface. Ddos mitigation: Why your traditional security fails. 2024, [Accessed 16 August 2024].
- [21] Rustam F, Jurcut AD, Aljedaani W, Ashraf I. Securing multi-environment networks using versatile synthetic data augmentation technique and machine learning algorithms. In: 2023 20th annual international conference on privacy, security and trust. 2023, p. 1–10.
- [22] Rustam F, Ranaweera PS, Jurcut AD. AI on the defensive and offensive: Securing multi-environment networks from AI agents. In: *IEEE ICC* 2024. 2024.
- [23] Aslam M, Ye D, Tariq A, Asad M, Hanif M, Ndzi D, et al. Adaptive machine learning based distributed denial-of-services attacks detection and mitigation system for SDN-enabled IoT. *Sensors* 2022;22(7):2697.
- [24] Data M, Aritsugi M. T-DFNN: An incremental learning algorithm for intrusion detection systems. *IEEE Access* 2021;9:154156–71.
- [25] Wu Z, Gao P, Cui L, Chen J. An incremental learning method based on dynamic ensemble RVM for intrusion detection. *IEEE Trans Netw Serv Manag* 2022;19(1):671–85.
- [26] Cai S, Tang H, Chen J, Hu Y, Guo W. CDDA-MD: An efficient malicious traffic detection method based on concept drift detection and adaptation technique. *Comput Secur* 2025;148:104121.
- [27] Mahdi ZS, Zaki RM, Alzubaidi L. A secure and adaptive framework for enhancing intrusion detection in IoT networks using incremental learning and blockchain. *Secur Priv* 2025;8(4):e70071.
- [28] Wang S-T, Sun S-S. An incremental learning model for network intrusion detection systems. In: 2024 10th international conference on applied system innovation. 2024, p. 362–4.
- [29] Mao J, Wei Z, Li B, Zhang R, Song L. Towards ever-evolution network threats: A hierarchical federated class-incremental learning approach for network intrusion detection in IIoT. *IEEE Internet Things J* 2024. 1–1.
- [30] Du L, Gu Z, Wang Y, Wang L, Jia Y. A few-shot class-incremental learning method for network intrusion detection. *IEEE Trans Netw Serv Manag* 2024;21(2):2389–401.
- [31] Tcydenova E, Kim T, Lee C, Park J. Detection of adversarial attacks in AI-based intrusion detection systems using explainable AI. *Human-Centric Comput Inf Sci* 2021.
- [32] Keshk M, Koroniotis N, Pham N, Moustafa N. An explainable deep learning-enabled intrusion detection framework in IoT networks. *Inform Sci* 2023.
- [33] Sauka K, Shin G, Kim D, Han M. Adversarial robust and explainable network intrusion detection systems based on deep learning. *Appl Sci* 2022;12(13):6451.
- [34] Lundberg H, Mowla N, Abedin S, Thar K. Experimental analysis of trustworthy in-vehicle intrusion detection system using explainable AI. In: *IEEE conference on communications and network security*. IEEE; 2022.
- [35] Arreche O, Guntur T, Abdallah M. XAI-IDS: Toward proposing an explainable artificial intelligence framework for enhancing network intrusion detection systems. *Appl Sci* 2024;14(10):4170.
- [36] Sharma B, Sharma L, Lal C, Roy S. Explainable artificial intelligence for intrusion detection in IoT networks: A deep learning based approach. *Expert Syst Appl* 2024;238:121751.
- [37] Gan X, Duanmu J, Wang J, Cong W. Anomaly intrusion detection based on PLS feature extraction and core vector machine. *Knowl-Based Syst* 2013;46:1–8.
- [38] Ankam D, Bouguila N. Compositional data analysis with PLS-DA and security applications. In: 2018 IEEE conference on information reuse and integration. IEEE; 2018, p. 398–405.
- [39] Thakkar A, Kikani N, Geddam R. Fusion of linear and non-linear dimensionality reduction techniques for feature reduction in LSTM-based intrusion detection system. *Appl Soft Comput* 2024;111378.
- [40] Ahmed A. Year. Year. IoT-23 Preprocessed Data, Accessed on: Date, <https://www.kaggle.com/datasets/engraqeel/iot23preprocesseddata>.
- [41] Sharafaldin I, Lashkari AH, Hakak S, Ghorbani AA. Developing realistic distributed denial of service (ddos) attack dataset and taxonomy. In: 2019 international carnahan conference on security technology. 2019, p. 1–8.
- [42] Bifet A, Gavalda R. Learning from time-changing data with adaptive windowing. In: *Proceedings of the 2007 SIAM international conference on data mining*. SIAM; 2007, p. 443–8.
- [43] Sakurada M, Yairi T. Anomaly detection using autoencoders with nonlinear dimensionality reduction. In: *Proceedings of the MLSDA 2014 2nd workshop on machine learning for sensory data analysis*. 2014, p. 4–11.
- [44] Jaworski M, Rutkowski L, Angelov P. Concept drift detection using autoencoders in data streams processing. In: *International conference on artificial intelligence and soft computing*. Springer; 2020, p. 124–33.
- [45] Liu Y, Wang J, Li J, Niu S, Song H. Class-incremental learning for wireless device identification in IoT. *IEEE Internet Things J* 2021;8(23):17227–35.
- [46] Constantinides C, Shiaeles S, Ghita B, Kolokotronis N. A novel online incremental learning intrusion prevention system. In: 2019 10th IFIP international conference on new technologies, mobility and security. IEEE; 2019, p. 1–6.
- [47] Li Y, Yuan Y. Convergence analysis of two-layer neural networks with relu activation. *Adv Neural Inf Process Syst* 2017;30.
- [48] Daubechies I, DeVore R, Foucart S, Hanin B, Petrova G. Nonlinear approximation and (deep) ReLU networks. *Constr Approx* 2022;55(1):127–72.
- [49] Baldi P, Sadowski PJ. Understanding dropout. *Adv Neural Inf Process Syst* 2013;26.
- [50] Franke M, Degen J. The softmax function: Properties, motivation, and interpretation. 2023.
- [51] Chen Z, Liu B. Continual learning and catastrophic forgetting. In: *Lifelong machine learning*. Cham: Springer International Publishing; 2018, p. 55–75.
- [52] Kozal J, Wasilewski J, Krawczyk B, Woźniak M. Continual learning with weight interpolation. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2024, p. 4187–95.
- [53] Parmar A, Kataria R, Patel V. A review on random forest: An ensemble classifier. In: *International conference on intelligent data communication technologies and internet of things*. Springer; 2019, p. 758–63.
- [54] Bahad P, Saxena P. Study of adaboost and gradient boosting algorithms for predictive analytics. In: *International conference on intelligent computing and smart communication 2019: Proceedings of ICSC 2019*. Springer; 2020, p. 235–44.
- [55] Suthaharan S, Suthaharan S. Support vector machine. *Mach Learn Model Algorithms Big Data Classif: Think Examples Eff Learn* 2016;207–35.
- [56] Field A. Logistic regression. *Discov Stat using SPSS* 2009;264(1):315.
- [57] Liao Y, Vemuri VR. Use of k-nearest neighbor classifier for intrusion detection. *Comput Secur* 2002;21(5):439–48.
- [58] Azimjonov J, Kim T. Stochastic gradient descent classifier-based lightweight intrusion detection systems using the efficient feature subsets of datasets. *Expert Syst Appl* 2024;237:121493.
- [59] Hossain MR, Timmer D. Machine learning model optimization with hyper parameter tuning approach. *Glob J Comput Sci Technol D Neural Artif Intell* 2021;21(2):31.
- [60] Bergstra J, Bengio Y. Random search for hyper-parameter optimization. *J Mach Learn Res* 2012;13(2).
- [61] Sarwar A, Mushtaq MF, Akram U, Rustam F, Hamza A, Rupapara V, et al. IoT networks attacks detection using multi-novel features and extra tree random-voting ensemble classifier (ER-VEC). *J Ambient Intell Humaniz Comput* 2023;14(12):16637–51.
- [62] Ridgeway G. Generalized boosted models: A guide to the gbm package. *Update* 2007;1(1):2007.
- [63] Han S, Qubo C, Meng H. Parameter selection in SVM with RBF kernel function. In: *World automation congress 2012*. IEEE; 2012, p. 1–4.
- [64] Reeves D, Jacyna G. Support vector machine regularization. *Wiley Interdiscip Rev: Comput Stat* 2011;3(3):204–15.
- [65] Wazirali R. An improved intrusion detection system based on KNN hyperparameter tuning and cross-validation. *Arab J Sci Eng* 2020;45(12):10859–73.
- [66] Moore R, DeNero J. L1 and L2 regularization for multiclass hinge loss models. In: *MLSLP*. 2011, p. 1–5.
- [67] Brightwood S. Regularization techniques in logistic regression. *Ladoke Akintola Univ Technol March* 2024.
- [68] M VM, Khullar V, Bhosle AA, Salunke MD, Bangare JL, Ingavale A. Streamed incremental learning for cyber attack classification using machine learning. In: 2022 2nd international conference on innovative sustainable computational technologies. 2022, p. 1–5.
- [69] Beshah YK, Abebe SL, Melaku HM. Drift adaptive online ddos attack detection framework for IoT system. *Electronics* 2024;13(6):1004.
- [70] Ferrag MA, Friha O, Hamouda D, Maglaras L, Janicke H. Edge-IIoTset: A new comprehensive realistic cyber security dataset of IIoT and IIoT applications: Centralized and federated learning. 2022.
- [71] Salih AM, Raisi-Estabragh Z, Galazzo IB, Radeva P, Petersen SE, Lekadir K, et al. A perspective on explainable artificial intelligence methods: SHAP and LIME. *Adv Intell Syst* 2025;7(1):2400304.
- [72] Data4V. An introduction to interpretable machine learning with LIME and SHAP. 2024, [Accessed 03 February 2025].